Ethical considerations

Spotify genre classifier · ML in Production Environment

> **In short:** This app guesses the genre of a song from how it sounds, and it is open about how sure it is. But the model behind it is weak. It only gets 81.7% of the test songs right, and it does much worse on music that is different from what it was trained on, like Indian music and songs that mix genres. It can give a useful hint, but it should not decide what genre a song "really" is, and it should never be used to make decisions about artists.

Data

The model is trained on a Kaggle dataset of Spotify songs. The full dataset has 114 genres, but I only use six of them: classical, country, heavy-metal, hip-hop, jazz and techno. After cleaning, 4,650 songs are left. Of these, 3,720 are used for training and 930 for testing.

The genre labels are not facts. Each song has just one label, which shows how it was grouped when the data was collected. Many songs fit more than one genre, but I removed the songs that were listed under more than one of the six genres. This made the data cleaner, but it also means the model has never seen a song that mixes genres, even though those are very common in real music.

The genres are also mostly Western. The full dataset has genres like indian, k-pop and samba, but this model only knows six genres from American and European music. Any other music is pushed into one of the six, or gets "not sure". Even the "classical" label mixes very different traditions: Mozart is next to Indian classical singers like Bhimsen Joshi. Later in the report I show what this does to the results.

The genres are not the same size either. There are 991 heavy-metal songs but only 500 jazz songs, about half as many, and jazz is also the genre the model does worst on. On top of that, songs by the same artist can be in both the training set and the test set. This makes the test results look better than they really are.

The model

The model is XGBoost, which combines many small decision trees. It uses 14 features. Thirteen of them describe the sound, like acousticness, danceability and tempo. The last one, "explicit", only says if the lyrics are explicit.

I left popularity out on purpose. Popularity measures how much a song is played, not how it sounds. If the model used it, it would partly learn which genres are popular, and a new artist with few plays could get a different answer just because nobody knows them yet. Without popularity, the accuracy went down from 88.1% to 81.7%. I think that is worth it, since the tool should judge the sound and not the fame.

The "explicit" feature is a problem. It is about the lyrics, not the sound, and it mostly points to one genre: 25.2% of the hip-hop songs are explicit, but none of the classical songs are. This can teach the model that explicit lyrics mean hip-hop, which is a stereotype. It is also the least important of the 14 features in the permutation test, so removing it would cost almost nothing.

The XGBoost settings come from a RandomizedSearchCV search in the first part of the project, when popularity was still included. They may not be the best settings for the model without popularity, so the next step would be to tune the model again and test it on artists it has never seen.

Threshold tuning: when should the app say "not sure"?

For every song, the model gives each genre a probability, which is how likely it thinks that genre is. A normal model just picks the genre with the highest probability, even if it is only 30%. This app works differently. Every genre has its own threshold, which is the lowest probability the genre needs before the app picks it. If no genre reaches its threshold, the app says "not sure". If more than one does, the one with the highest probability wins.

To choose the thresholds, I tested 19 values for each genre, from 0.05 to 0.95, and kept the one with the best F1 score. F1 is a score from 0 to 1. It is only high when the model finds most of the songs in a genre and is also right most of the times it names that genre. I did this with 5-fold cross-validation on the training songs, so every song was predicted by a model that had not seen it. The test songs were not used at all, which keeps the final test fair. Table 1 shows the thresholds and how each genre does on the test songs.

Table 1: The tuned thresholds and the results on the 930 test songs. Precision is how often the app is right when it names the genre. Recall is how many of the genre's songs it finds.

Genre	Threshold	Test songs	Precision	Recall	F1	"Not sure"
classical	0.35	161	0.900	0.894	0.897	1
country	0.30	127	0.687	0.724	0.705	4
heavy-metal	0.55	198	0.920	0.869	0.894	8
hip-hop	0.45	149	0.826	0.732	0.776	10
jazz	0.35	100	0.678	0.610	0.642	6
techno	0.40	195	0.853	0.831	0.842	8

A good example is "justified" by Kacey Musgraves, which is labeled country. The model gives hip-hop the highest probability, 29.7%, but hip-hop needs 45%. Country only gets 16.9%, and no other genre reaches its threshold either. So instead of wrongly saying hip-hop, the app says "not sure". This happened for 37 of the 930 test songs.

So what did the thresholds change? When the app gives an answer, it is right more often: 740 of its 893 answers are right (82.9%), compared to 81.7% without thresholds. But if "not sure" counts as a wrong answer, the accuracy goes down from 81.7% to 79.6%, and the macro F1 (the average F1 of the six genres) goes from 0.797 to 0.793. So the thresholds don't make the model smarter. They make it more careful.

This raises a few ethical questions.

"Not sure" doesn't hit all genres the same. Saying "not sure" is more honest than a wrong guess, but it is not spread evenly. 10 of the 149 hip-hop songs (6.7%) and 6 of the 100 jazz songs (6.0%) got "not sure", but only 1 of the 161 classical songs (0.6%). It happens when the probabilities are spread over several genres, which is typical for songs that mix styles, like "justified".

Different genres have different rules. A song needs 55% to be called heavy-metal, but only 30% to be called country, and this changes who gets the mistakes. Because the country threshold is low, the app now calls 134 songs country instead of 126, and more of them are wrong: the precision for country drops from 0.722 to 0.687. Heavy-metal is the opposite. With its high threshold, the app is right more often when it says heavy-metal (the precision goes from 0.889 to 0.920), but it finds fewer of the real heavy-metal songs (the recall goes from 0.934 to 0.869). Every threshold is a choice between two mistakes: giving a song the wrong genre, or missing a song that belongs to the genre. F1 treats both mistakes as equally bad. In a real product, a person should make that choice on purpose.

The numbers can be made to look better than they are. If every threshold is set to 0.70, the app is right 629 of the 685 times it answers (91.8%). That sounds great until you see that it says "not sure" for 245 of the 930 songs. This is why the app counts "not sure" as wrong in its accuracy, and always shows how many songs got "not sure".

One change affects everyone. The sliders make the model easier to understand, because users see the results change right away. But the thresholds are stored in the backend, so when one user changes them, the answers change for every user. There is no login to stop this, and the changes are lost when the backend restarts.

Feature importance

I used two methods to see which features the model depends on. Permutation importance shuffles one feature at a time in the test songs and measures how much worse the model gets. The bigger the drop, the more the model needs that feature. XGBoost's total gain measures how much each feature helped the trees split the songs during training. Both methods agree on the top three: acousticness (0.203 in the permutation test and 0.243 in total gain), danceability (0.116 and 0.137) and instrumentalness (0.089 and 0.114).

This shows what the model "listens" to, but it tells us about the model, not about music. A song isn't jazz because of its acousticness score. Features that go together can also share the importance between them. Energy and loudness have a correlation of 0.83, so each of them can look less important than it really is. Both methods also only describe the model as a whole. The app can't explain why one specific song got its genre, which is what a user would need to judge a single answer.

Why this is a weak model

At first the test results look okay, but a closer look shows that the model is weak in several ways.

It is wrong quite often. Without thresholds it gets 170 of the 930 test songs wrong (18.3%). With thresholds, 153 answers are wrong and 37 are "not sure".

It does worse on artists it has never seen. 796 of the 930 test songs (85.6%) are by artists who also have songs in the training set, and the app gets 646 of them right (81.2%). For the 134 songs by new artists, it only gets 94 right (70.1%). So for new music, the main results are too optimistic.

It can be very sure and still be wrong. Of the 486 test songs where the model was at least 90% sure, 14 were wrong. One of them is "The Champion" by Carrie Underwood and Ludacris. It is labeled country, but the model is 96.9% sure it is hip-hop. Ludacris raps on the song, so it is really both. Many of the confident mistakes are like this: country songs with rap in them, or songs with a doubtful label, like reggaeton labeled as hip-hop.

It knows very little about each song. The model never hears the music. It only gets 14 numbers per song, and nothing about the lyrics, the instruments or where the music comes from, even though a genre is also about culture and history. These numbers are Spotify's own estimates, made by another algorithm, so the model is built on guesses. In November 2024, Spotify also stopped giving these numbers to new apps, so the model can't easily be used on new songs.

The task was made easy. I picked the six genres because they sound very different and share few songs. Real music has many genres that are close to each other (the full dataset has 114, like rock, alt-rock and indie), and that would be a much harder task.

It is also uneven between genres. The F1 score is 0.897 for classical and 0.894 for heavy-metal, but only 0.705 for country and 0.642 for jazz (see Table 1).

Who gets the mistakes?

The mistakes are not spread evenly. Most of them land on music that doesn't fit into the six boxes. Table 2 shows how often the app gives the right genre for some groups of test songs.

Table 2: How often the app gives the right genre for different groups of test songs ("not sure" counts as wrong). I sorted the classical songs into Western and Indian by the artist's name.

Group of test songs	Songs	Right genre
Western songs labeled classical	123	121 (98.4%)
Indian songs labeled classical	38	23 (60.5%)
Songs by artists who are also in the training set	796	646 (81.2%)
Songs by new artists	134	94 (70.1%)
All test songs	930	740 (79.6%)

The biggest gap is for Indian music. Of the 161 classical test songs, 123 are Western and 38 are Indian (some of these are really film or devotional songs). The app gets 121 of the Western songs right, but only 23 of the Indian ones, so 15 of the 17 classical songs it misses are Indian. This is probably because the model learned what "classical" means mostly from Western music, which is mostly instrumental, while a lot of the Indian music is sung. Even my first notebook described classical as "mostly orchestral and instrumental music", which is a Western way of seeing it.

Songs that mix genres are also hit. Country songs with rap, like "The Champion", get called hip-hop with high confidence, and mixed songs are also the ones that most often get "not sure". New artists get the right genre less often than artists the model already knows (70.1% against 81.2%), even though new and small artists are the ones who need a fair label the most. And jazz and country have the lowest F1 scores, so artists in those genres get more wrong labels.

If a music service used a model like this to put songs on playlists, these groups would be the ones most often shown to the wrong listeners. The problem could also grow by itself: a song that is shown to the wrong listeners gets fewer plays, looks less popular, and then gets shown even less.

Usability and accessibility

The app is one page with three views: Predict, Thresholds and Features. Users can search for a song or describe one themselves with sliders. For songs from the dataset, the prediction is shown next to the real label, so it is easy to see when the model is wrong. Probability bars with a line for each threshold show how sure the model is, and a "not sure" answer comes with a short explanation, for example "Not sure: hip-hop is most likely (30%), but below its threshold." Extra details are hidden until the user opens them.

Album covers, music videos and a DJ mode make the app fun to use, but they can also take attention away from the model's limits. The moving background stops for people who have turned off animations on their computer, but the light grey text on the dark glass design can be hard to read for people with poor eyesight. The app also only knows six genres, can only judge songs from the dataset or songs described with the sliders, and needs two servers to start. So it works as a demo, not as a product for people who aren't technical.

Trustworthiness: when to trust it, and when not

The app does several things to earn trust. It shows the test results for the current thresholds (79.6% accuracy and a macro F1 of 0.793 with the tuned thresholds). It shows the real label next to each prediction, warns when a song was used in training (because then the result isn't a fair test), and says "not sure" instead of guessing. The backend also checks the input: missing or wrong values give an error, and thresholds outside 0 to 1 are ignored. Still, none of this fixes doubtful labels or the missing genres.

Some things work against trust. There is no login, so anyone who can reach the backend can change the thresholds for everyone. Album covers, song previews and videos come from Spotify, Apple, YouTube and Wikidata. The app has no control over these services. A video found by search can be the wrong one, and the services can see which songs a user looks up. The polished design, with album covers, videos and glass effects, also makes the model look more professional than it is. People already tend to trust computers too much (this is called automation bias), and a good-looking app makes that worse.

So when can we trust an AI model? In general, we can trust it more when:

- mistakes are cheap and easy to fix,
- the input is similar to what the model was trained on,
- the model says how sure it is, and that has been checked,
- a person can check the answer, and
- the model has been tested fairly on new cases.

We should not trust it when the stakes are high, when the input is very different from the training data, or when the question has no single right answer, like what genre a mixed song "really" is.

For this model, the good news is that its confidence means something. Table 3 shows that when the model is at least 90% sure, its top genre is right 97.1% of the time. When it is less than 50% sure, it is right only 44.3% of the time.

Table 3: How often the model's top genre is right, depending on how sure the model is (930 test songs, without thresholds).

How sure the model is	Test songs	Top genre right
Less than 50%	97	43 (44.3%)
50% to under 70%	148	88 (59.5%)
70% to under 90%	199	157 (78.9%)
90% or more	486	472 (97.1%)

This means the model can be trusted for learning how sound features relate to genres, and as a first guess for clear, typical songs of the six genres when it is very sure. It should not be trusted for songs that mix genres, for music outside the six genres or from non-Western traditions, for songs where it is less than 70% sure, or for new artists without someone checking.

Most important, it should never be used to make decisions about people, like which playlists artists get on, how much they are paid, or how they are ranked and promoted. A model can't be held responsible for its mistakes, so a person always has to make these decisions and take responsibility for them.

The good and the bad

To sum up, these are the good sides:

- Popularity is left out, so fame doesn't decide the genre.
- The app says "not sure" instead of guessing, and counts "not sure" as wrong in its accuracy, so the numbers are not made to look better.
- The thresholds were chosen without using the test songs, so the test is fair.
- The app is open about itself. It shows the test results, the real label, the thresholds and the features the model uses, and it warns when a song was used in training.
- The model's confidence means something. When it is at least 90% sure, it is right 97.1% of the time.
- It uses no personal data about listeners and has no user accounts.

And these are the bad sides:

- The labels are opinions, mixed songs were removed, and the genres are Western. Only 23 of the 38 Indian songs labeled classical get the right genre, against 121 of the 123 Western ones.
- The model is weak. It gets 18.3% of the test songs wrong, and only 70.1% right for new artists.
- It can be very sure and still be wrong.
- The "explicit" feature links hip-hop to explicit lyrics.
- One user can change the thresholds for everyone, there is no login, and the changes are lost when the backend restarts.
- Outside services can see which songs users look up.
- The polished design can make the model seem more trustworthy than it is.

> **Conclusion:** The model works best as a helpful hint for clear examples of the six genres, when it is very sure. It should not have the final word on mixed, culturally specific or unusual music, and it should never be used to make decisions about artists. A better version would need more and fairer data (including mixed and non-Western music), testing on new artists, explanations for single songs, and a login so one user can't change the settings for everyone.